

**Hugo de Freitas Evangelista, Paulo Henrique Calisto dos Santos, Hudson
Junior Rodrigues Silva, Ivan Aloizio de Lana Filho**

**Cardly: aplicativo móvel de flashcards com revisão
espaçada e integração cliente-servidor**

**Ipatinga – MG
2026**

**Hugo de Freitas Evangelista, Paulo Henrique Calisto dos Santos, Hudson
Junior Rodrigues Silva, Ivan Aloizio de Lana Filho**

**Cardly: aplicativo móvel de flashcards com revisão espaçada
e integração cliente-servidor**

Trabalho final apresentado na disciplina de Sistemas Móveis do curso de Engenharia de Software do Centro Universitário do Leste de Minas Gerais – UNILESTE, Campus Coronel Fabriciano, como requisito parcial para avaliação da referida disciplina.

Centro Universitário do Leste de Minas Gerais – UNILESTE, Campus Coronel Fabriciano

Orientador: Prof. Daniel Rocha Gualberto

Ipatinga – MG
2026

RESUMO

O Cardly é um aplicativo multiplataforma de flashcards com repetição espaçada, inspirado no Anki, voltado ao estudo de diferentes disciplinas com acompanhamento de progresso. O front-end foi desenvolvido com React Native, Expo e TypeScript; o back-end, com Spring Boot, Java e PostgreSQL. O sistema integra autenticação por e-mail, senha e Google SSO, CRUD de disciplinas e cartões, estudo e revisão com agendamento automático, comunidade, rede de amigos, notificações e painel administrativo. Este relatório apresenta contextualização, metodologia, objetivos, requisitos, arquitetura, trechos de código, testes e conclusões, em conformidade com os critérios da disciplina e com as normas ABNT.

Palavras-chave: flashcards. revisão espaçada. React Native. Spring Boot. arquitetura cliente-servidor.

LISTA DE QUADROS

1	Matriz de Requisitos Funcionais	10
2	Matriz de Requisitos Não Funcionais	11

SUMÁRIO

Sumário	3
1 Introdução	4
1.1 Contextualização	4
1.2 Justificativa	4
1.3 Relevância da Solução Proposta	4
2 Metodologia	6
2.1 Processo de Engenharia de Software	6
2.2 Justificativa do Stack Tecnológico	6
2.3 Estratégia de Validação e Coleta de dados	6
3 Objetivos	8
3.1 Objetivo Geral	8
3.2 Objetivos Específicos	8
4 Requisitos do Sistema	9
4.1 Requisitos Funcionais	9
4.2 Requisitos Não Funcionais	9
5 Design e Arquitetura	12
5.1 Visão Lógica e de Componentes	12
5.2 Visão de Dados	12
5.2.1 Diagrama de dados	13
5.3 Visão Física e de Implantação	14
6 Explicação de Trechos Importantes de Código	15
7 Testes e Validação	17
7.1 Cenários de Teste	17
7.2 Visão do Usuário	17
7.2.1 Figuras	17
7.3 Visão do Administrador	17
7.3.1 Figuras	18
8 Conclusão	25
Referências	26

1 INTRODUÇÃO

1.1 Contextualização

A memorização de longo prazo exige revisões distribuídas no tempo, princípio consolidado pela repetição espaçada e popularizado por ferramentas como o Anki (Anki Project, 2024; SUPERMEMO, 2022). Estudantes de Engenharia de Software precisam dominar múltiplas disciplinas simultaneamente, mas muitas soluções genéricas não integram estudo mobile, progresso visual e compartilhamento de conteúdo em um único ecossistema (PRESSMAN; MAXIM, 2020; SOMMERVILLE, 2019).

A disciplina de Sistemas Móveis exige demonstração prática de integração entre front-end React Native e back-end Spring Boot, com autenticação, CRUD e comunicação via API REST (Meta Platforms, 2025; VMware Spring, 2025; TANENBAUM; WETHERALL; FEAMSTER, 2015). O Cardly nasce nesse contexto: um projeto acadêmico, sem fins comerciais, que replica funcionalidades centrais de flashcards e revisão espaçada.

1.2 Justificativa

Flashcards são adequados para avaliar integração completa porque combinam regras de negócio não triviais (agendamento de revisões, sessões de estudo e revisão distintas, visibilidade pública/privada de disciplinas) com demandas de UX mobile (animação de virada de cartão, feedback imediato, dashboard) (Anki Project, 2024; NGUYEN, 2023; W3Schools, 2024).

A equipe optou por stack amplamente documentada — Expo, TypeScript, Spring Boot e PostgreSQL — para reduzir risco em um time do sétimo semestre com pouca experiência prévia em desenvolvimento mobile (Expo, 2025b; ROCKETSEAT, 2024; SILVA; OVERFLOW, 2024). Como sugestão complementar do professor, o grupo também preparou o sistema para deploy em nuvem, embora isso não fosse requisito obrigatório da avaliação (WIGGINS, 2022).

1.3 Relevância da Solução Proposta

O Cardly oferece estudo individual com repetição espaçada por máquina de estados (NONE, MEDIUM, HARD, EASY) e intervalos fixos de 2h, 4h e 36h, comunidade para clonar disciplinas públicas, rede de amigos por código numérico, notificações de revisão vencida e solicitações pendentes, além de painel SUPERADMIN para gestão global. A solução demonstra arquitetura cliente-servidor, segurança com JWT e OAuth Google, organização em camadas (Controller, Service, Repository no back-end; telas, hooks e serviços no front-end) e documentação formal em ABNT (Associação Brasileira de Normas Técnicas, 2011; Auth0, 2024; Google, 2025).

O código-fonte está disponível nos repositórios GitHub do projeto (<<https://github.com/hugoFreit4s/cardly-backend>> e <<https://github.com/hugoFreit4s/cardly-rnative-app>>).

2 METODOLOGIA

2.1 Processo de Engenharia de Software

O desenvolvimento seguiu ciclos iterativos inspirados em entrega incremental: cada entrega consolidou um conjunto coerente de funcionalidades — autenticação, CRUD, estudo, comunidade, amigos, notificações e Google SSO — com commits distribuídos entre os integrantes da equipe. Requisitos foram levantados a partir do escopo da disciplina, do documento inicial *Flashcards.pdf* e de validação contínua contra o comportamento do sistema (PRESSMAN; MAXIM, 2020; SOMMERVILLE, 2019).

Documentação técnica (regras de negócio e este relatório) foi mantida no repositório cardly-docs, separado dos repositórios de código. Decisões de domínio foram registradas em migrations Flyway no back-end e validadas por testes automatizados (Redgate, 2024; JUnit Team, 2024).

2.2 Justificativa do Stack Tecnológico

Front-end: React Native com Expo e TypeScript permite um único código para Android, iOS e web, acelerando prototipação e builds via EAS (Meta Platforms, 2025; Expo, 2025b; Microsoft, 2025; Expo, 2025a). NativeWind trouxe estilização utilitária semelhante ao Tailwind; React Navigation estruturou fluxos autenticado/não autenticado (NativeWind Team, 2025; React Navigation Contributors, 2025).

Back-end: Spring Boot 3 oferece ecossistema maduro para APIs REST, segurança, JPA e testes (VMware Spring, 2025; Spring Security Community, 2024; Spring Data JPA Team, 2024). PostgreSQL atende persistência relacional com integridade referencial; Flyway versiona o esquema (PostgreSQL Global Development Group, 2025; Redgate, 2024).

Infraestrutura (opcional): Como extensão além do escopo mínimo, a equipe publicou uma versão demonstrativa em nuvem (Docker, EC2 e RDS), facilitando a apresentação presencial; trata-se de sugestão adotada voluntariamente, não exigência da disciplina (Docker Inc., 2025; Amazon Web Services, 2025; WIGGINS, 2022).

2.3 Estratégia de Validação e Coleta de dados

A validação combinou três frentes: (1) **testes automatizados** no back-end (JUnit e MockMvc) para autenticação, cartões, amigos e notificações; (2) **testes manuais** guiados pelos critérios de aceite das tabelas de requisitos; (3) **demonstração funcional** integrada (login, estudo, admin), com capturas de tela no Capítulo 7 (JUnit Team, 2024).

Coleta de feedback foi informal entre os integrantes durante sprints semanais. Limitações conhecidas (calendário estilo *heatmap* parcial, ausência de refresh token) foram registradas na conclusão para transparência na apresentação.

3 OBJETIVOS

3.1 Objetivo Geral

Desenvolver e documentar o Cardly, aplicativo móvel multiplataforma de flashcards integrado a API REST, demonstrando domínio da arquitetura cliente-servidor exigida na disciplina de Sistemas Móveis, com autenticação segura, CRUD completo, regras de revisão espaçada e funcionalidades sociais (Meta Platforms, 2025; VMware Spring, 2025; TANENBAUM; WETHERALL; FEAMSTER, 2015).

3.2 Objetivos Específicos

- Implementar autenticação com JWT, cadastro por e-mail/senha e login com Google SSO, incluindo papéis USER e SUPERADMIN (Auth0, 2024; Google, 2025; Spring Security Community, 2024).
- Modelar e expor CRUD de disciplinas e cartões, sessões de estudo/revisão, comunidade, amigos e notificações via endpoints REST (Spring Data JPA Team, 2024; springdoc-openapi Project, 2024).
- Construir interface React Native com flip animado, dashboard, calendário de atividade e fluxos de administração, garantindo usabilidade em mobile e web (Software Mansion, 2025; NativeWind Team, 2025; W3Schools, 2024).
- Validar o sistema com testes automatizados no back-end e cenários manuais alinhados aos requisitos funcionais (JUnit Team, 2024).

4 REQUISITOS DO SISTEMA

4.1 Requisitos Funcionais

4.2 Requisitos Não Funcionais

ID	Descrição	Critério de Aceite
RF-01	O sistema deve permitir cadastro de usuário com e-mail e senha.	Conta criada; e-mail único; senha entre 8 e 128 caracteres.
RF-02	O sistema deve permitir login com e-mail e senha.	Token JWT retornado; credenciais inválidas retornam erro genérico.
RF-03	O sistema deve permitir login com conta Google (SSO).	E-mail verificado; conta existente reativada ou nova conta criada.
RF-04	O usuário deve poder excluir a própria conta.	Soft delete; disciplinas e cartões do usuário também excluídos logicamente.
RF-05	O usuário deve gerenciar disciplinas (criar, editar, excluir, reordenar).	CRUD persistido; dono exclusivo da disciplina.
RF-06	O usuário deve definir disciplina como pública ou privada.	Disciplinas públicas visíveis na comunidade.
RF-07	O usuário deve gerenciar cartões (criar, editar, excluir).	Cartão vinculado à disciplina do usuário; acesso via Gerenciar cartões em Minhas Disciplinas.
RF-08	O usuário deve responder cartões em sessão de estudo ou revisão.	Resposta aplica transição por dificuldade atual (NONE/MEDIUM/HARD/EASY) com intervalos 2h, 4h ou 36h; cartão some da fila até vencer.
RF-09	O usuário deve poder pular cartão sem marcar acerto/erro.	Cartão reagendado conforme dificuldade/intervalo atuais.
RF-10	O usuário deve visualizar progresso no dashboard.	Métricas de estudo, streak e calendário de dias com atividade.
RF-11	O usuário deve enviar, aceitar, recusar e cancelar solicitações de amizade.	Amizade via código público de 6 dígitos; estados PENDING/ACCEPTED/DENIED.
RF-12	O usuário deve clonar disciplinas públicas da comunidade.	Cópia independente na coleção do usuário.
RF-13	O usuário deve receber notificações de revisão vencida e amizades.	Lista, marcar lida, excluir e som opcional no app.
RF-14	O SUPERADMIN deve gerenciar usuários, disciplinas e cartões globalmente.	Painel admin com CRUD e busca paginada.

Fonte: Elaborado pelos autores (2026).

Quadro 1: Matriz de Requisitos Funcionais

ID	Categoria	Restrição / Métrica
RNF-01	Desempenho	Respostas da API adequadas ao uso simultâneo em demonstração acadêmica.
RNF-02	Segurança	Senhas com BCrypt; JWT HS256 com expiração de 1 h; rotas protegidas por filtro Spring Security.
RNF-03	Configurabilidade	Variáveis de ambiente para URL da API e credenciais; preparação opcional para deploy em nuvem.
RNF-04	Portabilidade	Front-end único em React Native/Expo para Android, iOS e web.
RNF-05	Manutenibilidade	Migrations Flyway versionadas; camadas Controller/Service/Repository no back-end.
RNF-06	Usabilidade	Feedback visual (toasts, modais); flip de cartão acionado por toque; ícones e navegação consistentes.
RNF-07	Documentação	Relatório ABNT, regras de negócio e referências bibliográficas rastreáveis.

Fonte: Elaborado pelos autores (2026).

Quadro 2: Matriz de Requisitos Não Funcionais

5 DESIGN E ARQUITETURA

5.1 Visão Lógica e de Componentes

A arquitetura segue o padrão cliente-servidor em três camadas no back-end e organização por features no front-end (TANENBAUM; WETHERALL; FEAMSTER, 2015; PRESSMAN; MAXIM, 2020).

Back-end (Spring Boot):

- **Controllers** — expõem endpoints REST (/api/auth, /api/decks, /api/cards, /api/friends, /api/notifications, /api/admin).
- **Services** — concentram regras de negócio (agendamento de revisão espaçada, soft delete, notificações agendadas).
- **Repositories** — acesso JPA ao PostgreSQL; Specifications para filtros paginados (Spring Data JPA Team, 2024).
- **Security** — filtro JWT, UserDetailsService, OAuth Google e roles (Spring Security Community, 2024; Auth0, 2024).

Front-end (React Native/Expo):

- **Navegação** — stacks autenticado/não autenticado com React Navigation (React Navigation Contributors, 2025).
- **Telas** — login, dashboard, disciplinas, detalhe da disciplina (cartões), estudo, comunidade, amigos, notificações, admin.
- **Serviços/Hooks** — cliente HTTP com token, contexto de autenticação e configuração Google em runtime (Mozilla Developer Network, 2024).
- **AdminScreen** — listagens de usuários e disciplinas implementadas com FlatList, componente nativo do React Native para listas roláveis com boa performance (Meta Platforms, 2025).

5.2 Visão de Dados

Principais entidades e relacionamentos:

- **User** — credenciais, papel (USER/SUPERADMIN), código público, soft delete.
- **Deck (disciplina)** — pertence a um usuário; flag public; ordem de exibição.

- **Card** — pergunta/resposta, dueAt, streaks, dificuldade e intervalo agendado.
- **FriendRequest** — remetente, destinatário, status e timestamps.
- **Notification** — tipo, payload JSON, lida/não lida, usuário destino.
- **StudyReview** — histórico de respostas para dashboard e calendário.

Migrations Flyway (V1–V11) evoluem o esquema de forma incremental (Redgate, 2024; PostgreSQL Global Development Group, 2025).

5.2.1 Diagrama de dados

Para documentação do modelo relacional, foi preparado um script DBML compatível com dbdiagram.io em `cardly-docs/assets/diagrama-dados-dbdiagram.dbml`. A captura gerada na plataforma pode ser inserida na figura abaixo.

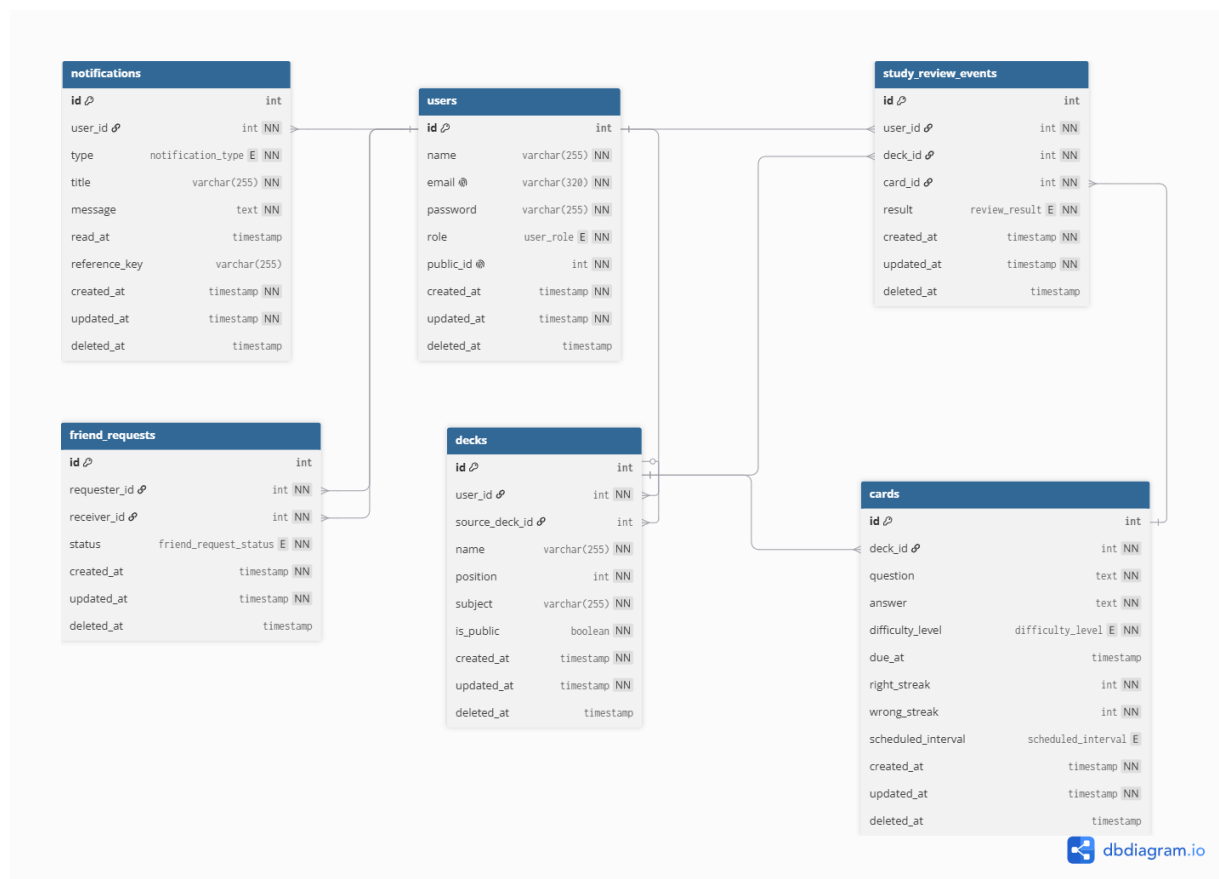


Figura 1 – Diagrama de dados do Cardly

Fonte: Elaborado pelos autores (2026).

5.3 Visão Física e de Implantação

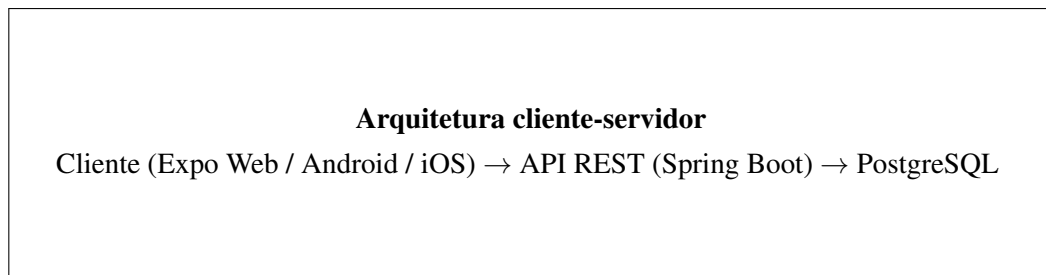


Figura 2 – Visão física simplificada do Cardly

Fonte: Elaborado pelos autores (2026).

Em ambiente de desenvolvimento, front-end e back-end rodam localmente (Expo + Spring Boot embarcado). Opcionalmente, a equipe publicou uma instância em nuvem para facilitar a demonstração na apresentação — recurso extra, não exigido pelo professor (WIGGINS, 2022; Docker Inc., 2025).

6 EXPLICAÇÃO DE TRECHOS IMPORTANTES DE CÓDIGO

O agendamento de revisão é a regra de negócio central do Cardly: define quando o cartão volta à fila após acerto, erro ou skip. O trecho abaixo, extraído do CardService, implementa a máquina de estados adotada na versão v12 e registra a revisão para o dashboard (Anki Project, 2024; NGUYEN, 2023; VMware Spring, 2025).

Listagem 6.1 – Transição de dificuldade após resposta do usuário

```
1  @Transactional
2  public CardResponse answerCard(Card card, AnswerCardRequest request
3      ) {
4      if (request.correct()) {
5          card.setRightStreak(card.getRightStreak() + 1);
6          card.setWrongStreak(0);
7      } else {
8          card.setRightStreak(0);
9          card.setWrongStreak(card.getWrongStreak() + 1);
10     }
11
12     ScheduleTransition t = transitionForAnswer(
13         card.getDifficultyLevel(), request.correct());
14     applySchedule(card, t.difficulty(), t.interval());
15
16     Card saved = cardRepository.save(card);
17     studyReviewService.registerReview(saved,
18         request.correct() ? ReviewResultEnum.CORRECT :
19         ReviewResultEnum.WRONG);
20     return toResponse(saved);
21 }
22
23 private ScheduleTransition transitionForAnswer(
24     DifficultyLevelEnum level, boolean correct) {
25     return switch (level) {
26         case NONE -> correct
27             ? new ScheduleTransition(MEDIUM, HOURS_4)
28             : new ScheduleTransition(HARD, HOURS_2);
29         case MEDIUM -> correct
30             ? new ScheduleTransition(EASY, HOURS_36)
```



```

29         : new ScheduleTransition(HARD, HOURS_2);
30     case HARD -> correct
31         ? new ScheduleTransition(MEDIUM, HOURS_4)
32         : new ScheduleTransition(HARD, HOURS_2);
33     case EASY -> correct
34         ? new ScheduleTransition(EASY, HOURS_36)
35         : new ScheduleTransition(MEDIUM, HOURS_4);
36     };
37 }

```

Explicação técnica: o método é transacional para garantir consistência entre atualização do cartão e registro histórico. O streak continua sendo atualizado a cada resposta (acertos consecutivos em `rightStreak` e erros consecutivos em `wrongStreak`), porém o intervalo é definido exclusivamente pela matriz de transição por dificuldade atual (NONE, MEDIUM, HARD, EASY). Esse desenho mantém previsibilidade dos intervalos e facilita validação por testes automatizados. O controller correspondente exige JWT válido e verifica ownership do cartão antes de delegar ao serviço (Spring Security Community, 2024; SILVA; OVERFLOW, 2024).

No front-end, a sessão de estudo consome GET `/api/cards/duo` e envia POST `/api/cards/{id}/answer` ou `/skip`, atualizando a fila local após cada interação (Meta Platforms, 2025; Mozilla Developer Network, 2024).

7 TESTES E VALIDAÇÃO

7.1 Cenários de Teste

Os cenários abaixo foram executados manualmente no ambiente de desenvolvimento e homologação, complementados por testes automatizados no back-end quando aplicável (JUnit Team, 2024).

1. **Autenticação** — cadastro, login e-mail/senha, login Google, token expirado retorna 401.
2. **Disciplinas e cartões** — CRUD completo em Minhas Disciplinas (botão **Gerenciar cartões**); disciplina pública aparece na comunidade; clone gera cópia independente; excluir a cópia e voltar à Comunidade restaura o botão **Clonar**.
3. **Estudo** — validar matriz de transição: NONE + acerto = MEDIUM + 4h, NONE + erro = HARD + 2h, MEDIUM + acerto = EASY + 36h, MEDIUM + erro = HARD + 2h, além de transições de HARD e EASY.
4. **Amigos** — solicitação por código #123456; aceitar/recusar/cancelar atualiza estado e notificações.
5. **Notificações** — scheduler gera alerta de revisão vencida; usuário marca lida ao tocar.
6. **Admin** — SUPERADMIN lista/edita/exclui usuários e conteúdo de terceiros.

7.2 Visão do Usuário

As capturas a seguir ilustram o fluxo do usuário comum: autenticação, dashboard, notificações e sessão de estudo. Legendas e fontes seguem o padrão ABNT (Associação Brasileira de Normas Técnicas, 2011).

7.2.1 Figuras

7.3 Visão do Administrador

O painel SUPERADMIN utiliza o componente `FlatList` do React Native nas listagens de usuários e de disciplinas, garantindo rolagem eficiente mesmo com muitos registros retornados pela API administrativa (Meta Platforms, 2025). As figuras abaixo referem-se exclusivamente a essa visão gerencial.

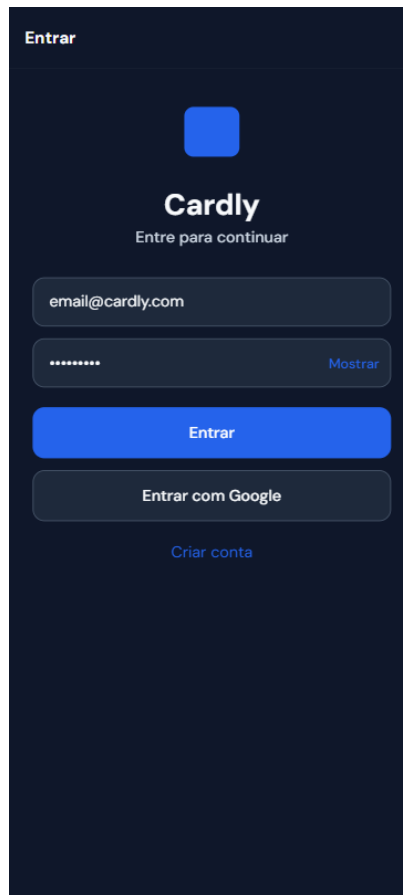


Figura 3 – Tela de autenticação do Cardly (e-mail, senha e Google SSO)

Fonte: Elaborado pelos autores (2026).

7.3.1 Figuras

As capturas administrativas foram obtidas durante a validação funcional do painel SUPERADMIN, com legendas e fontes no padrão ABNT (Associação Brasileira de Normas Técnicas, 2011).

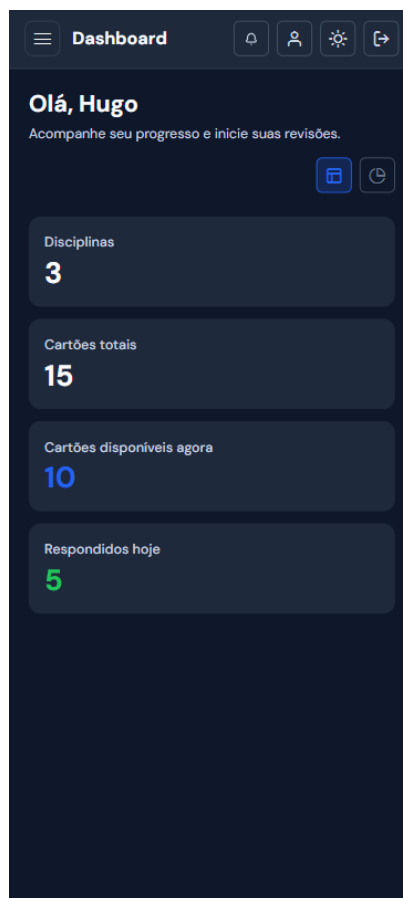


Figura 4 – Dashboard do usuário — visão geral de progresso (sem gráficos)

Fonte: Elaborado pelos autores (2026).



Figura 5 – Dashboard do usuário — visão com gráficos de desempenho

Fonte: Elaborado pelos autores (2026).

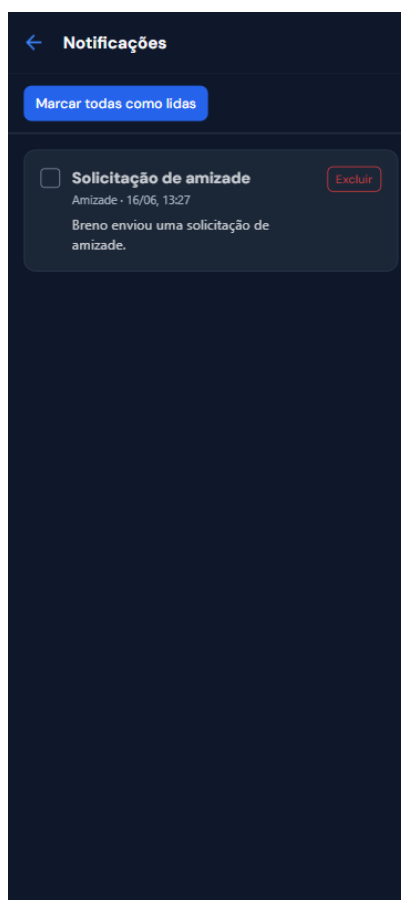


Figura 6 – Tela de notificações (revisões vencidas e solicitações de amizade)

Fonte: Elaborado pelos autores (2026).

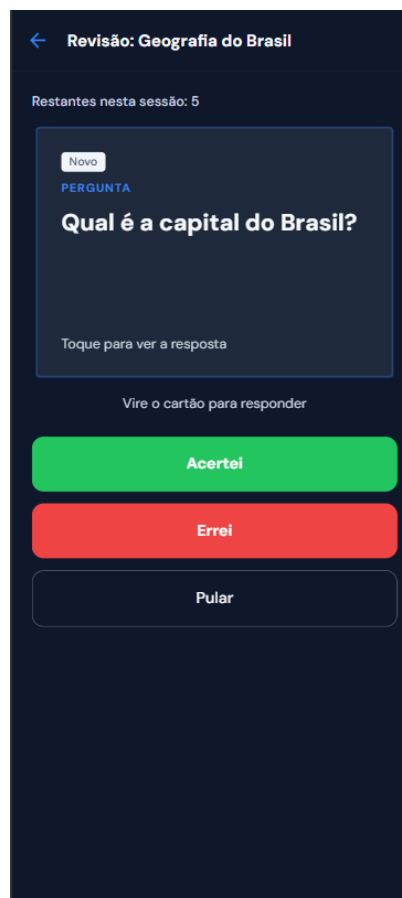


Figura 7 – Sessão de estudo — visualização e interação com cartão (flip)

Fonte: Elaborado pelos autores (2026).

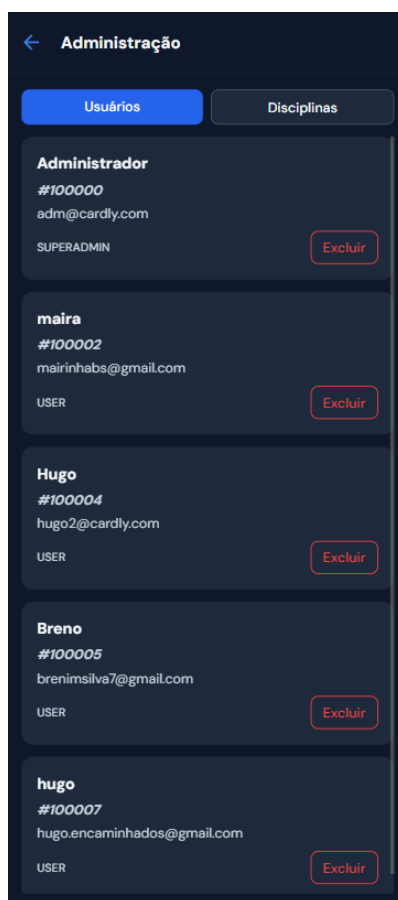


Figura 8 – Painel administrativo — listagem de usuários (FlatList)

Fonte: Elaborado pelos autores (2026).

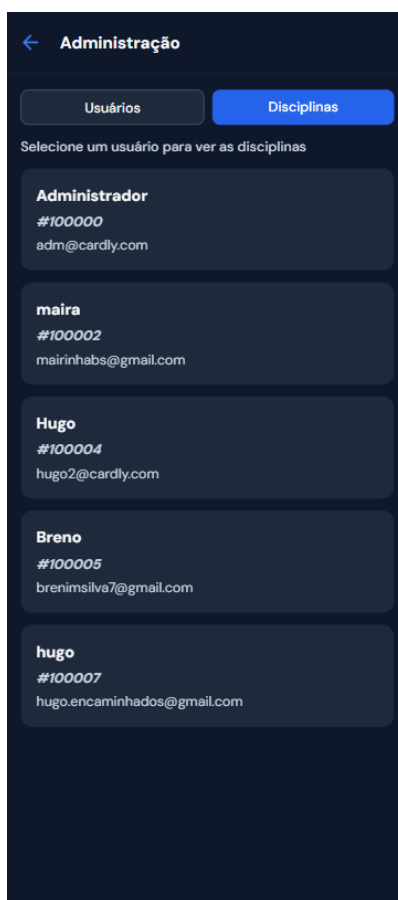


Figura 9 – Painel administrativo — listagem de disciplinas (FlatList)

Fonte: Elaborado pelos autores (2026).

8 CONCLUSÃO

O Cardly atingiu o objetivo de integrar front-end React Native (Expo/TypeScript) e back-end Spring Boot (Java) em arquitetura cliente-servidor, com autenticação JWT e Google SSO, CRUD de disciplinas e cartões, revisão espaçada, comunidade, amigos, notificações e painel administrativo (Meta Platforms, 2025; VMware Spring, 2025).

A documentação formal segue a estrutura aprovada pelo professor (capa, resumo, requisitos em quadros, arquitetura, código, testes e referências ABNT), com fonte Times New Roman conforme manual UNILESTE (Associação Brasileira de Normas Técnicas, 2011; Associação Brasileira de Normas Técnicas, 2018). A demonstração prática cobre login, navegação, CRUD, API protegida e regras de agendamento de revisão — alinhada aos critérios de avaliação da disciplina.

Gargalos e limitações: calendário visual ainda simplificado em relação ao design inspirado em apps de ciclo; ausência de refresh token (sessão expira em 1 h); cobertura automatizada limitada no front-end mobile; rate limiting e observabilidade avançada não implementados. Esses pontos devem ser explicitados na arguição.

Trabalhos futuros: heatmap mensal completo no dashboard; notificações push nativas (Expo Notifications); refresh token; testes E2E com Detox/Maestro; documentação OpenAPI interativa; modo escuro; sincronização offline (Expo, 2025b; springdoc-openapi Project, 2024; Clue by Biowink, 2024).

A equipe consolidou aprendizado em JWT end-to-end, animações com Reanimated, Specification JPA e listagens performáticas com FlatList no painel admin — competências centrais para projetos mobile integrados exigidos no sétimo semestre (FIRESHIP, 2022; Software Mansion, 2025; Spring Data JPA Team, 2024; Meta Platforms, 2025).

Código-fonte: <<https://github.com/hugoFreit4s/cardly-backend>> e <<https://github.com/hugoFreit4s/cardly-rnative-app>>.

REFERÊNCIAS

Amazon Web Services. *AWS Free Tier*. 2025. Disponível em: <<https://aws.amazon.com/free/>>. Acesso em: 4 jun. 2026.

Anki Project. *Anki Manual*. 2024. Documentação oficial sobre baralhos, cartas e algoritmos de revisão. Disponível em: <<https://docs.ankiweb.net/>>. Acesso em: 5 jun. 2026.

Associação Brasileira de Normas Técnicas. *NBR 14724: Informação e documentação – Trabalhos acadêmicos – Apresentação*. Rio de Janeiro: ABNT, 2011. Norma para formatação de relatórios e monografias.

Associação Brasileira de Normas Técnicas. *NBR 6023: Informação e documentação – Referências – Elaboração*. Rio de Janeiro: ABNT, 2018. Norma para elaboração de referências bibliográficas.

Auth0. *JSON Web Token Introduction*. 2024. Conceitos de claims, assinatura e fluxo stateless. Disponível em: <<https://jwt.io/introduction>>. Acesso em: 30 maio 2026.

Clue by Biowink. *Clue Period Tracker*. 2024. Referência de UX para calendário de atividade diária. Disponível em: <<https://helloclue.com/>>. Acesso em: 31 maio 2026.

Docker Inc. *PostgreSQL Official Image*. 2025. Ambiente local de banco de dados para desenvolvimento. Disponível em: <https://hub.docker.com/_/postgres>. Acesso em: 30 maio 2026.

Expo. *EAS Build*. 2025. Geração de APK/AAB para demonstração em dispositivo físico. Disponível em: <<https://docs.expo.dev/build/introduction/>>. Acesso em: 6 jun. 2026.

Expo. *Expo Documentation*. 2025. Build, desenvolvimento e publicação com Expo SDK. Disponível em: <<https://docs.expo.dev/>>. Acesso em: 29 maio 2026.

FIRESHIP. *JWT Authentication Explained*. 2022. Disponível em: <<https://www.youtube.com/watch?v=7Q17ubqLfaM>>. Acesso em: 1 jun. 2026.

Google. *Using OAuth 2.0 for Web Server Applications*. 2025. Disponível em: <<https://developers.google.com/identity/protocols/oauth2/web-server>>. Acesso em: 1 jun. 2026.

JUnit Team. *JUnit 5 User Guide*. 2024. Disponível em: <<https://junit.org/junit5/docs/current/user-guide/>>. Acesso em: 5 jun. 2026.

Meta Platforms. *React Native Documentation*. 2025. Guia oficial de componentes, navegação e integração nativa. Disponível em: <<https://reactnative.dev/docs/getting-started>>. Acesso em: 29 maio 2026.

Microsoft. *TypeScript Handbook*. 2025. Disponível em: <<https://www.typescriptlang.org/docs/handbook/intro.html>>. Acesso em: 29 maio 2026.

Mozilla Developer Network. *Using the Fetch API*. 2024. Base conceitual para chamadas HTTP no front-end. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch>. Acesso em: 2 jun. 2026.

NativeWind Team. *NativeWind Documentation*. 2025. Estilização com utilitários Tailwind em React Native. Disponível em: <<https://www.nativewind.dev/>>. Acesso em: 2 jun. 2026.

NGUYEN, L. *Spaced Repetition for Developers: A Practical Guide*. 2023. Artigo introdutório sobre intervalos de revisão e retenção. Disponível em: <<https://medium.com/@linhnguyen/spaced-repetition-for-developers>>. Acesso em: 30 maio 2026.

PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. Disponível em: <<https://www.postgresql.org/docs/>>. Acesso em: 30 maio 2026.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: uma abordagem profissional*. 9. ed. Porto Alegre: McGraw Hill, 2020.

React Navigation Contributors. *React Navigation Documentation*. 2025. Disponível em: <<https://reactnavigation.org/docs/getting-started>>. Acesso em: 31 maio 2026.

Redgate. *Flyway Documentation*. 2024. Versionamento de schema com migrations SQL. Disponível em: <<https://documentation.red-gate.com/fd/flyway-documentation-138346877.html>>. Acesso em: 30 maio 2026.

ROCKETSEAT. *React Native com Expo e TypeScript – fundamentos mobile*. 2024. Série de vídeos utilizada pela equipe para consolidar base mobile. Disponível em: <https://www.youtube.com/results?search_query=react+native+expo+typescript>. Acesso em: 29 maio 2026.

SILVA, R.; OVERFLOW comunidade S. *How to implement JWT authentication in Spring Boot 3?* 2024. Discussões práticas sobre filtros JWT e SecurityFilterChain. Disponível em: <<https://stackoverflow.com/questions/tagged/spring-boot+jwt>>. Acesso em: 1 jun. 2026.

Software Mansion. *React Native Reanimated Documentation*. 2025. Disponível em: <<https://docs.swmansion.com/react-native-reanimated/>>. Acesso em: 2 jun. 2026.

SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.

Spring Data JPA Team. *Specifications*. 2024. Disponível em: <<https://docs.spring.io/spring-data/jpa/reference/jpa/specifications.html>>. Acesso em: 3 jun. 2026.

Spring Security Community. *OAuth 2.0 Resource Server JWT*. 2024. Disponível em: <<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>>. Acesso em: 1 jun. 2026.

springdoc-openapi Project. *OpenAPI 3 Documentation for Spring Boot*. 2024. Documentação automática de endpoints REST. Disponível em: <<https://springdoc.org/>>. Acesso em: 3 jun. 2026.

SUPERMEMO. *SuperMemo Algorithm SM-2*. 2022. Referência histórica para algoritmos de repetição espaçada. Disponível em: <<https://www.supermemo.com/en/archives1990-2015/english/ol/sm2>>. Acesso em: 30 maio 2026.

TANENBAUM, A. S.; WETHERALL, D. J.; FEAMSTER, N. *Redes de Computadores*. 6. ed. São Paulo: Pearson, 2015.

VMware Spring. *Spring Boot Reference Documentation*. 2025. Disponível em: <<https://docs.spring.io/spring-boot/reference/index.html>>. Acesso em: 30 maio 2026.

W3Schools. *How To Create a Flip Card with CSS*. 2024. Inspiração visual para animação de virada de carta. Disponível em: <https://www.w3schools.com/howto/howto_css_flip_card.asp>. Acesso em: 29 maio 2026.

WIGGINS, A. *The Twelve-Factor App*. 2022. Boas práticas para aplicações prontas para produção. Disponível em: <https://12factor.net/pt_br/>. Acesso em: 4 jun. 2026.